

# Igra Protocol: Leaderless Two-Phase Coordination with CRDT-Based Threshold Signing

*Authors to be determined*  
*Affiliations to be determined*

January 20, 2026

## Abstract

Igra is a leaderless protocol for coordinating threshold signature groups on UTXO-based blockchains. The protocol addresses the fundamental problem of maintaining safety when multiple independent signers must agree on a single transaction template despite observing potentially inconsistent UTXO sets from their respective blockchain nodes. We present a two-phase coordination mechanism combined with CRDT-based signature propagation that ensures at most one transaction is signed per event, even under crash faults and temporary network partitions. The protocol guarantees safety under crash-fault assumptions with non-equivocating signers, and we provide formal correctness proofs for unique commitment, deterministic agreement, and CRDT convergence. We validate the theoretical model through a reference implementation deployed on the Kaspa blockchain.

## 1 Introduction

### 1.1 The UTXO Coordination Problem

**UTXO model background.** In UTXO-based blockchains (Bitcoin, Kaspa, Litecoin, etc.), transactions consume specific unspent transaction outputs (UTXOs) as inputs and create new outputs [1]. Unlike account-based models (Ethereum), UTXO chains lack global shared state and smart contract programmability for coordination logic [2]. Each node independently maintains its own UTXO set view, and temporary inconsistencies arise from:

- Network propagation delays between geographically distributed nodes
- Temporary chain reorganizations (forks) before probabilistic finality
- Node synchronization lag during initial sync or after downtime
- Conflicting transactions pending in different mempools
- Variable block confirmation times (Kaspa: 1 sec, Bitcoin: 10 min)

**Lack of programmability.** UTXO chains, by design, do not support Turing-complete smart contracts at the base layer [3]. Bitcoin Script and Kaspa TxScript are intentionally limited: they provide signature verification and time-locks but not general computation or persistent state. This means coordination logic *cannot be implemented on-chain*. Proposed solutions like covenants [4] and extended UTXO models [2] (Cardano) add expressiveness but do not solve the fundamental distributed coordination problem for threshold signers operating across independent nodes.

**Multi-dimensional coordination problem.** When  $N$  independent signers must coordinate to produce an  $m$ -of- $N$  threshold signature on a payment transaction, they face *multiple sources of divergence*:

1. **UTXO set divergence:** Each signer queries their local blockchain node and may observe a *different UTXO set*. Even with eventual consistency, signers may build templates at different times and select different UTXOs as inputs.
2. **Event ordering divergence:** Payment events arrive from external sources (bridges, oracles, APIs) via asynchronous delivery. Signers may observe events in *different orders*: signer  $A$  sees  $(e_0, e_1, e_2)$  while signer  $B$  sees  $(e_1, e_0, e_2)$ . If events  $e_0$  and  $e_1$  could both spend the same UTXO, signing them in different orders leads to different UTXO availability and thus different templates.
3. **Concurrent event contention:** Multiple simultaneous events may compete for the same UTXOs. Without coordination, signers may assign a UTXO to different events, fragmenting signatures across incompatible templates.
4. **Temporal skew:** Signers process events at different wall-clock times due to processing delays, network latency, or crash-recovery cycles. A signer joining late may see an already-spent UTXO as available, proposing an invalid template.

If signers independently build transaction templates without coordination, they will almost certainly select different UTXOs, producing incompatible templates and fragmenting the signature set.

**Safety vs. liveness trade-off.** This creates a fundamental dilemma:

- **Optimistic approach:** Sign immediately without coordination. *Risk:* Signature fragmentation (liveness failure) when templates diverge. Worse: if some signers later observe updated UTXO sets and re-sign alternate templates, the group may sign *multiple distinct transactions* spending different UTXOs (double-spend, safety violation).
- **Conservative approach:** Wait for perfect UTXO convergence and event ordering agreement. *Risk:* Under prolonged divergence, high event throughput, or network partitions, convergence may never occur (indefinite liveness failure).

**Why existing approaches are insufficient.** 1. *Leader-based coordination (Raft, Paxos, PBFT with rotating leader):*

Leader-based protocols [5, 6] elect a single coordinator responsible for UTXO selection and template construction. While conceptually simple, this introduces several critical problems:

- **Single point of failure:** If the leader crashes, the system halts until a new leader is elected. Leader election itself requires multiple rounds of consensus, adding latency [5]. For high-frequency payment systems, this is unacceptable.
- **Single point of compromise:** A compromised leader can censor events, delay payments, or propose malicious templates. Even with eventual detection and leader replacement, attacks during the leader’s tenure can cause service disruption.

- **Per-event election overhead:** Running leader election per event adds  $O(\log N)$  message rounds [6] and coordination delay before every transaction. This is particularly costly for systems handling thousands of events per hour.
- **Per-time-frame coordination complexity:** Batching events into epochs with one leader per epoch requires:
  - Synchronizing epoch boundaries across asynchronously-arriving events
  - Buffering events, reducing responsiveness
  - Handling epoch transitions (leader changes mid-batch)
  - Resolving conflicts when epochs overlap or UTXO availability changes between epochs
- **Degraded performance under faults:** As noted in recent consensus surveys [7], leader-based protocols degrade significantly when leaders are slow, malicious, or crash. Leaderless approaches avoid this performance cliff.
- **Reduced decentralization:** Concentrating coordination power in a single leader contradicts the decentralization ethos of threshold signature schemes, where security relies on distributing trust across  $N$  independent parties [8].

## 2. *On-chain coordination via smart contracts:*

Using the destination UTXO chain itself for coordination (e.g., writing proposed templates to the blockchain) creates circular dependencies:

- Requires threshold signatures to write coordination state to the chain
- But coordination is needed to agree on which transaction to sign
- UTXO chains lack persistent contract state for multi-round protocols [3]
- High on-chain costs (every proposal = on-chain transaction)

## 3. *Centralized coordinator service:*

Delegating coordination to a trusted centralized service:

- Reintroduces trust assumptions beyond the cryptographic threshold scheme
- Single point of failure and compromise
- Requires out-of-band mechanisms to ensure coordinator honesty
- Defeats the purpose of decentralized threshold signing

## 4. *Optimistic execution without coordination:*

Signing templates immediately without agreement [9]:

- Works only if all signers observe identical UTXO sets simultaneously
- Fails under divergence: partial signatures scatter across incompatible templates
- No mechanism to recover from fragmentation
- Risk of double-spend if signers re-sign on updated views

**Robustness requirements.** A practical threshold signing system for UTXO chains must achieve:

1. **Safety:** At most one transaction is ever signed per payment request, even under UTXO divergence and event reordering.
2. **Liveness:** The system makes progress when possible (sufficient honest signers online, eventual UTXO convergence).
3. **Resilience:** Tolerate crash failures, message loss, temporary network partitions, and event ordering differences.
4. **Decentralization:** No single point of failure; all signers participate equally in coordination.
5. **Security:** Resist replay attacks, Sybil attacks, message forgery, and event injection.
6. **Auditability:** Explicit success/failure states; deterministic event-to-transaction mapping.

## 1.2 Design Principle

Igra’s fundamental invariant is:

**Single-signature per event:** Each signer signs at most one transaction template per unique event identifier.

This ensures that even if coordination fails, safety is preserved—no double-spending can occur.

## 1.3 Protocol Overview

Igra combines two protocols:

1. **Two-Phase Coordination:** Peers vote on transaction template hashes in a proposing phase, then commit to the quorum-winning hash before signing.
2. **CRDT Signature Propagation:** After commitment, partial signatures are propagated via conflict-free replicated data types, enabling any peer to finalize and submit once threshold is reached.

## 1.4 System Architecture

**Key abstractions.**

- **Validator set:** Any external attestation system with threshold signatures (examples: Hyperlane m-of-n validators, LayerZero endpoints, custom oracles).
- **UTXO chain:** Any blockchain using UTXO model (examples: Kasper, Bitcoin, Litecoin).
- **Gossip network:** Authenticated P2P coordination without centralized infrastructure.

## 1.5 Contributions

- A leaderless coordination protocol for UTXO threshold signing that guarantees safety under crash faults
- Formal proofs of unique commitment and CRDT convergence
- A reference implementation on Kasper blockchain validating the theoretical model

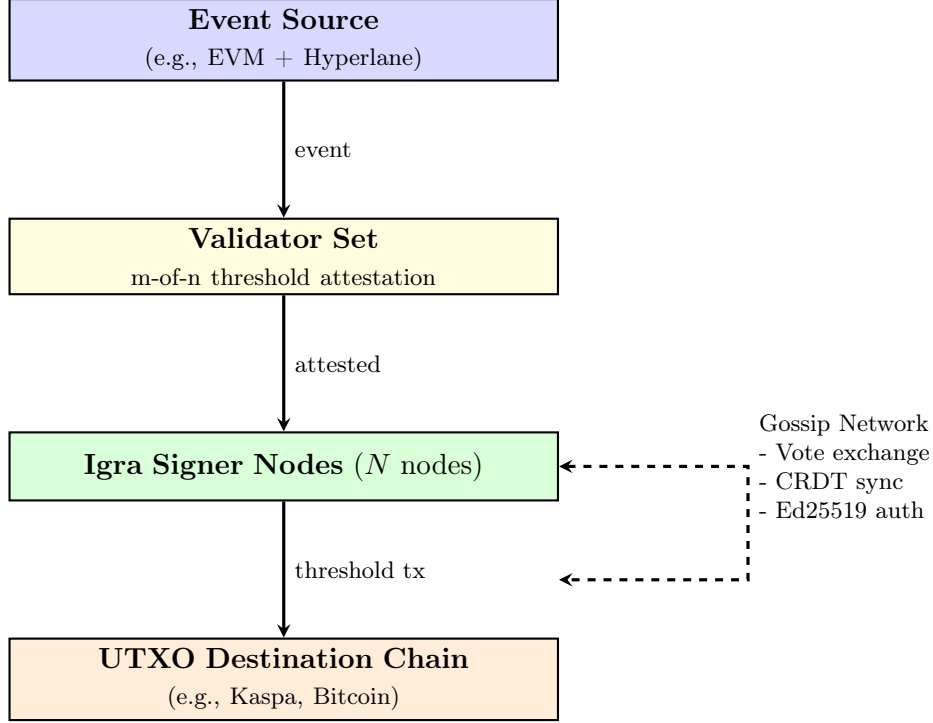


Figure 1: Igra system architecture. Events from external sources are attested by validator sets (key abstraction), processed by Igra nodes coordinating via authenticated gossip, and result in threshold-signed transactions on UTXO chains.

## 2 System Model and Trust Assumptions

### 2.1 Entities and Roles

**Signers.** A fixed group of  $N$  peers. Each peer has a stable identifier  $p_i \in \mathcal{P}$  and participates in a gossip network.

**Blockchain.** Each signer maintains a connection to a blockchain node providing:

- UTXO query interface
- Transaction submission interface
- Block height/finality information

**Event Sources.** External systems that produce signing requests (e.g., cross-chain bridges, payment systems).

**Validators.** External entities (e.g., Hyperlane validator set, LayerZero endpoints) that attest to the authenticity of cross-chain events.

## 2.2 Trust Assumptions

### 2.2.1 Event Authenticity

**Core concept: Validator sets.** Igra accepts payment events from external sources (cross-chain bridges, payment systems, oracles, etc.) provided they are attested by a *trusted validator set*. The key abstraction is a set of public keys and a threshold verification rule, independent of the specific event source.

**Assumption.** Events are authenticated by cryptographic signatures from an external validator set configured at system deployment.

**Enforcement.** Each event includes cryptographic proofs (signatures) from validators. The protocol verifies these proofs before accepting an event.

**Validator models (examples):** *Hyperlane-style (m-of-n threshold):*

- Configure  $M$  validator public keys (secp256k1 ECDSA)
- Require  $\geq T$  valid signatures where  $T \leq M$
- Each validator independently signs the message identifier
- Rejection: If fewer than  $T$  signatures verify correctly

*LayerZero-style (single endpoint):*

- Configure endpoint public key (secp256k1 ECDSA)
- Require exactly one valid signature from the endpoint
- Rejection: If signature verification fails

*Custom oracle or payment provider:*

- Configure provider-specific public keys and threshold rules
- Implement signature verification logic for provider’s attestation format

**Trust boundary.** We trust that:

1. The configured validator set is honest (or at least  $T$  of  $M$  validators are honest)
2. Validators will not collude to create fraudulent events
3. Validator public keys in configuration are correct and belong to legitimate validators

This trust is *external to Igra*—it is inherited from the event source’s security model (e.g., Hyperlane’s economic security via staked validators and slashing).

**Why this is acceptable.** Cross-chain messaging protocols, oracles, and payment systems typically use economic security (staking), cryptoeconomic incentives (slashing), and redundancy (Byzantine quorums) to ensure honest attestation. Igra inherits and leverages these existing security mechanisms rather than reimplementing attestation infrastructure. The protocol is *validator-set agnostic*: any event source with threshold signature attestation can be integrated.

### 2.2.2 Key Management

**Assumption.** Each signer’s private key is securely generated, stored, and accessed only by authorized processes.

**Key derivation (industry standard):**

- BIP39 mnemonic phrases (12-24 words)
- BIP32 hierarchical deterministic (HD) derivation
- Optional BIP44/BIP84 derivation paths
- Schnorr-compatible key extraction (x-only public keys)

**Key storage:**

- Private keys encrypted with XChaCha20Poly1305 authenticated encryption
- Encryption key sourced from environment variable or secure file
- Keys exist in memory only during signing operations
- Memory zeroed after use (Rust `Zeroize` trait)

**Trust boundary.** We assume:

1. The encryption key (`WALLET_SECRET`) is kept secret
2. No unauthorized access to process memory during signing
3. No unauthorized access to encrypted key files
4. The operating system enforces process isolation

**Current limitation.** No hardware security module (HSM) support. Keys exist in process memory during signing, making them vulnerable if the process is compromised or memory is dumped.

**Why this is acceptable for threshold systems.** Even if one key is compromised, threshold  $m$  requires compromising  $m$  independent keys to forge signatures. The threshold property provides defense-in-depth.

### 2.2.3 Peer Authentication

**Assumption.** The gossip network consists only of authorized signers.

**Enforcement:**

- Each peer has an Ed25519 keypair for message authentication
- Static whitelist of trusted peer public keys configured at startup
- All gossip messages signed; signatures verified against whitelist
- Unauthenticated messages are rejected before processing

**Trust boundary.** We trust that:

1. Configured peer public keys are correct (map peer IDs to actual signers)
2. Peer private keys are kept secret
3. No peer private key has been compromised

**Sybil resistance.** The static whitelist prevents Sybil attacks—an attacker cannot inject fake peers without compromising a legitimate peer’s private key.

#### 2.2.4 Storage Integrity

**Assumption.** Persistent storage is not tampered with.

**Production storage.** RocksDB on local filesystem:

- Proposals, CRDT state, phase state persisted to disk
- No encryption at rest for database files
- File system permissions expected to prevent unauthorized access

**Trust boundary.** We trust that:

1. The operating system enforces file permissions
2. No unauthorized process can read/write RocksDB files
3. Database files are not corrupted or tampered with

**Testing storage.** In-memory storage used only for automated tests; never used in production deployments.

#### 2.2.5 Blockchain Node Trust

**Assumption.** Each signer’s local blockchain node provides eventually-consistent UTXO data.

**Trust model:**

- Nodes may temporarily disagree on UTXO sets (reorgs, sync lag)
- Nodes eventually converge to canonical chain state
- Nodes do not intentionally lie about UTXO availability

**Why this is acceptable.** The two-phase protocol is specifically designed to handle temporary UTXO divergence. If nodes permanently diverge, the protocol abandons the event rather than risking double-spend.

**Blockchain finality.** We trust the underlying blockchain’s consensus mechanism. For Kaspas, this is GHOSTDAG with probabilistic finality based on blue score depth.



## 2.3 Threat Model

We assume **crash-fault tolerance**:

- **Honest majority**: At least  $\lceil N/2 \rceil + 1$  signers are correct.
- **Non-equivocation**: Correct signers do not send conflicting proposals for the same  $(event, round)$  pair.
- **Crash failures**: Signers may crash and be temporarily unreachable.
- **Message reordering**: Network may delay, duplicate, or reorder messages.

Byzantine behavior is considered a limitation and discussed in Section 4.4.

## 2.4 Network Model

We assume **partial synchrony**:

- There exists an unknown Global Stabilization Time (GST) and bound  $\Delta$  such that messages broadcast by correct peers at time  $t \geq \text{GST}$  are delivered within time  $\Delta$ .
- Messages before GST may experience unbounded delay.
- Gossip ensures eventual delivery among connected correct peers.

## 2.5 Cryptographic Assumptions

The adversary is computationally bounded and cannot:

- Find hash collisions in BLAKE3 (128-bit security)
- Forge signatures without private keys:
  - Ed25519 for message envelopes (128-bit security)
  - secp256k1 Schnorr for blockchain transactions (128-bit security)
  - secp256k1 ECDSA for validator attestations (128-bit security)

## 2.6 Problem Statement

**Input.** A canonical event  $e$  describing:

- Unique identifier
- Destination (blockchain address/script)
- Amount to transfer

**Output.** At most one valid blockchain transaction  $T$  corresponding to event  $e$  that is signed by threshold signers and submitted to the blockchain.

**Safety requirement.** No two distinct transactions  $T_1 \neq T_2$  for the same event  $e$  are both signed by threshold signers.

**Liveness requirement.** If sufficient honest signers are online and their nodes converge on a consistent UTXO view, the event eventually completes or is explicitly abandoned.

## 3 Protocol Specification

### 3.1 Core Data Structures

#### 3.1.1 Event Identifier

An **event** is a canonical payment request. The event identifier is computed deterministically:

$$event\_id := \mathcal{H}(domain \parallel encode(e)) \quad (1)$$

where  $\mathcal{H}$  is a cryptographic hash function (BLAKE3),  $domain$  is a protocol version string, and  $encode$  serializes the event fields.

#### 3.1.2 Transaction Template

A **transaction template**  $\tau$  is an unsigned transaction specifying:

- Input set:  $\{(txid_j, index_j, amount_j)\}$  (UTXO references)
- Output set:  $\{(destination_k, amount_k)\}$
- Metadata: fees, lock times, etc.

The **template identifier** is:

$$template\_id := \mathcal{H}(canonical(\tau)) \quad (2)$$

where  $canonical$  produces a deterministic serialization of the template.

#### 3.1.3 Proposal (Vote)

A **proposal** is a vote for a specific template in a round:

$$\pi := (event\_id, r, h, \tau_{blob}, p, t) \quad (3)$$

where:

- $r \in \mathbb{N}$ : round number
- $h$ : template identifier (hash)
- $\tau_{blob}$ : serialized template
- $p \in \mathcal{P}$ : proposer peer id
- $t$ : timestamp

### 3.1.4 Phase State

Each peer maintains local phase state for each event:

$$\phi := (\textit{phase}, r, h_{\textit{canon}}, h_{\textit{own}}) \quad (4)$$

where:

- $\textit{phase} \in \{\text{Unknown}, \text{Proposing}, \text{Committed}, \text{Completed}, \text{Failed}, \text{Abandoned}\}$
- $r$ : current round
- $h_{\textit{canon}}$ : committed template hash (once locked)
- $h_{\textit{own}}$ : own proposed hash

### 3.1.5 CRDT State

For each  $(\textit{event\_id}, \textit{template\_id})$  pair, peers maintain a replicated CRDT:

$$\mathcal{S} := (\Sigma, C, M, B) \quad (5)$$

where:

- $\Sigma$ : G-Set of signature records, keyed by  $(\textit{input\_idx}, \textit{pubkey})$
- $C$ : LWW-Register for completion status
- $M$ : Optional event metadata
- $B$ : Optional template blob

## 3.2 Protocol Invariants

**Invariant 3.1** (Phase Monotonicity). *Peers never transition backward from terminal states **Completed** or **Abandoned**.*

**Invariant 3.2** (Single Vote per Round). *For any  $(\textit{event\_id}, r)$ , each peer  $p_i$  contributes at most one proposal.*

**Invariant 3.3** (Single Signature per Event). *Each peer signs at most one template hash per  $\textit{event\_id}$ .*

**Invariant 3.4** (Commit Irreversibility). *Once a peer records  $h_{\textit{canon}}$  for an event, it never changes to a different hash.*

## 3.3 Two-Phase Coordination Protocol

### 3.3.1 Phase 1: Proposing

Upon receiving an event  $e$ :

1. **Validate external proofs:** Verify validator signatures against configured public keys. Require threshold  $T$  valid signatures (Hyperlane) or single endpoint signature (LayerZero). Reject if validation fails.

2. **Check policy:** Verify amount limits, destination whitelist, velocity limits. Reject if policy violated.
3. Compute  $event\_id$
4. Query local blockchain node for available UTXOs
5. Build transaction template  $\tau$  using deterministic UTXO selection:
$$seed := \mathcal{H}(event\_id \parallel r) \tag{6}$$
6. Compute  $h := template\_id(\tau)$
7. Broadcast proposal  $\pi := (event\_id, r, h, \tau_{blob}, p_i, t)$

### 3.3.2 Proposal Validation

Upon receiving proposal  $\pi$  from peer  $p_j$ :

1. **Authenticate sender:** Verify Ed25519 signature on message envelope against  $p_j$ 's configured public key. Reject if invalid.
2. **Replay protection:** Check if  $(p_j, session\_id, seq\_no)$  already seen. Reject duplicates.
3. **Rate limiting:** Check per-peer message rate. Reject if limit exceeded.
4. **Structural bounds:** Template size  $\leq$  maximum
5. **Event consistency:** Recompute  $event\_id$  from included event data; verify match
6. **Template consistency:**  $h = template\_id(\tau_{blob})$
7. **Policy constraints:** Re-verify policy on included event
8. **External proofs:** Re-verify validator signatures on included event

### 3.3.3 Vote Storage and Equivocation Detection

Proposals are keyed by  $(event\_id, r, p)$ . If peer  $p$  sends two proposals with distinct hashes for the same  $(event\_id, r)$ , the second is rejected as equivocation.

**Enforcement.** Storage returns an **Equivocation** error; the conflicting proposal is logged but not counted in the vote set.

**Limitation.** A Byzantine peer can send conflicting proposals to different peers before gossip converges. The protocol assumes this does not occur (non-equivocating signers assumption).

### 3.3.4 Phase 2: Committed

**Commit rule.** Let  $V_h$  be the set of distinct proposers who voted for hash  $h$  in round  $r$ . A peer may commit to  $h$  iff:

$$|V_h| \geq q \tag{7}$$

where  $q$  is the commit quorum threshold.

**Canonical selection.** If multiple hashes satisfy quorum (impossible when  $q > N/2$ ), select deterministically:

1. Prefer hash with highest vote count
2. Tie-break by numerically smaller hash
3. Tie-break by lexicographically smaller proposer id
4. Among tied proposals, select by minimal deterministic score:

$$score := \mathcal{H}(domain \parallel event\_id \parallel r \parallel p) \quad (8)$$

**Lock.** Upon commitment, set  $h_{canon} := h$  and transition to **Committed** phase.

### 3.4 CRDT Signature Protocol

#### 3.4.1 Signature Generation

Once committed to template hash  $h$ , each peer:

1. Retrieves template  $\tau$  corresponding to  $h$
2. **Derive signing key:** Use BIP32 HD derivation from encrypted mnemonic
3. For each input  $i$  in  $\tau$ , computes signature  $\sigma_i$  over input digest
4. Broadcasts signature records to CRDT (via authenticated gossip)

**Key management trust.** The signing key is derived from an encrypted mnemonic stored locally. We assume:

- The mnemonic was generated with sufficient entropy (industry standard: 128-256 bits)
- The encryption key is kept secret
- No unauthorized access to process memory during signing

#### 3.4.2 CRDT Merge Semantics

**Signature Set (G-Set).** Merge is set union:

$$\Sigma \leftarrow \Sigma \cup \Sigma' \quad (9)$$

De-duplication by key ( $input\_idx, pubkey$ ) ensures at most one signature per signer per input.

**Completion (LWW-Register).** For completion records  $C = (txid, p, t, \dots)$  and  $C' = (txid', p', t', \dots)$ :

$$merge(C, C') := \begin{cases} C & \text{if } t \geq t', \\ C' & \text{otherwise.} \end{cases} \quad (10)$$

**Template Blob.** First valid non-empty value is kept (validated against  $h$ ).

### 3.4.3 Threshold Satisfaction

**Definition 3.5** (Threshold Predicate). Let  $m$  be the signature threshold and  $I$  the number of inputs. The threshold predicate holds iff:

$$\forall i \in \{0, \dots, I-1\} : |\{(\sigma, p) \mid (\sigma, p) \in \Sigma_i\}| \geq m \quad (11)$$

where  $\Sigma_i$  is the set of signatures for input  $i$ .

### 3.4.4 Finalization and Submission

When threshold is observed, any peer may:

1. Apply signatures from  $\Sigma$  to template  $\tau$
2. Finalize into signed transaction  $T$
3. Submit  $T$  to blockchain
4. Record completion:  $C := (txid(T), p_i, t_{now}, chain\_tip)$

**Concurrent finalization.** Multiple peers may finalize simultaneously. This is safe:

- Deterministic signature application produces identical  $T$
- Blockchain treats duplicate submissions as idempotent (same txid)
- LWW merge on completion ensures convergence

## 3.5 Fast-Forward Mechanism

A peer receiving a committed CRDT state can bypass local proposal construction by:

1. **Re-validating external proofs** on the included event
2. **Re-verifying template hash** against included template blob
3. **Re-checking policy** constraints

If all checks pass, jump directly to **Committed** phase. This supports late-joining peers without requiring trusted setup.

## 3.6 Anti-Entropy and Retries

**Anti-entropy.** Peers periodically sync CRDT state via request/response to repair missed messages.

**Rate limiting.** Sync requests are rate-limited per peer (10 messages/second sustained, 100 message burst).

**Retries.** If quorum not reached within timeout, increment round  $r := r+1$  and return to proposing phase. After bounded retries (configurable, typically 5-10 rounds), mark event as **Abandoned**.

## 4 Correctness Analysis

### 4.1 Safety: Unique Commitment

**Theorem 4.1** (Unique Quorum Hash). *Fix an event and round. Assume Invariant 3.2 (single vote per round) and  $q > N/2$ . Then at most one template hash  $h$  satisfies  $|V_h| \geq q$ .*

*Proof.* Assume for contradiction that distinct hashes  $h \neq h'$  both satisfy  $|V_h| \geq q$  and  $|V_{h'}| \geq q$ .

By Invariant 3.2, each peer votes once per round. A vote for  $h$  cannot simultaneously vote for  $h' \neq h$ , thus  $V_h \cap V_{h'} = \emptyset$ .

Total votes:  $|V_h| + |V_{h'}| \geq 2q > N$ , contradicting  $N$  total peers.  $\square$

**Theorem 4.2** (Agreement on Committed Hash). *Let peer  $A$  commit to hash  $h$  in round  $r$  based on proposal set  $P_r$ . Let peer  $B$  later observe  $P'_r \supseteq P_r$ . Under Invariant 3.2 and  $q > N/2$ , peer  $B$  selects the same hash  $h$ .*

*Proof.* Peer  $A$  committed to  $h$  with  $|V_h| \geq q$  in  $P_r$ . Since  $P'_r \supseteq P_r$ , all votes for  $h$  are present in  $P'_r$ , thus  $V_h \subseteq V'_h$  and  $|V'_h| \geq q$ .

By Theorem 4.1 applied to  $P'_r$ , no distinct  $h' \neq h$  satisfies quorum. The deterministic selection function filters to quorum hashes and selects among them canonically. Since only  $h$  qualifies in  $P'_r$ , peer  $B$  selects  $h$ .  $\square$

### 4.2 CRDT Convergence

**Theorem 4.3** (Signature Set Convergence). *Under eventual delivery, all correct peers converge to the same final signature set for a committed (event\_id, template\_id) pair.*

*Proof.* Let  $S_i(t)$  denote peer  $i$ 's signature set at time  $t$ .

Properties:

1. **Monotonicity:** G-Set never removes elements, thus  $S_i(t) \subseteq S_i(t')$  for  $t < t'$ .
2. **Commutativity/Associativity:** Merge is set union, which is commutative and associative.
3. **Idempotence:**  $S \cup S = S$ .
4. **Eventual delivery:** Every signature  $\sigma$  generated by a correct peer is eventually delivered to all correct peers.

By eventual delivery, every correct peer eventually includes all signatures from correct peers. Union's commutativity and associativity ensure convergence regardless of merge order:

$$S_{final} = \bigcup_{p \in correct} signatures(p) \quad (12)$$

Completion uses LWW-Register; under eventual delivery, all peers observe the maximal-timestamp record and converge.  $\square$

### 4.3 Liveness

**Property 4.4** (Bounded Termination). *Every event reaches either **Completed** or **Abandoned** within bounded time.*

**Justification.** The protocol uses explicit round timeouts and bounded retry counters. Each round either reaches quorum or times out. After maximum retries, the event is marked **Abandoned**.

**Non-guarantee.** Completion is not guaranteed under arbitrary UTXO divergence or prolonged network partitions. Safety is prioritized over liveness.

## 4.4 Security Analysis

### 4.4.1 Threats Mitigated

#### T1. UTXO Divergence.

- **Threat:** Peers observe different UTXO sets.
- **Mitigation:** Quorum voting prevents split commitment (Theorems 4.1, 4.2).

#### T2. Message Replay.

- **Threat:** Adversary replays old messages.
- **Mitigation:** Authenticated envelopes, storage deduplication (24-hour TTL), phase gating.

#### T3. Network Partition.

- **Threat:** Peers cannot reach quorum.
- **Outcome:** Protocol times out, abandons event (no unsafe signing).

#### T4. Fraudulent Events.

- **Threat:** Attacker injects fake cross-chain messages.
- **Mitigation:** Validator signature verification (m-of-n threshold for Hyperlane, single endpoint for LayerZero).

#### T5. Sybil Attacks.

- **Threat:** Attacker floods gossip with fake peers.
- **Mitigation:** Static whitelist of Ed25519 public keys; only authenticated peers participate.

### 4.4.2 Limitations

**A1. Byzantine Equivocation.** A Byzantine peer can send conflicting proposals to different peers before gossip converges. The protocol assumes this doesn't occur; full BFT requires explicit lock certificates.

**A2. Denial of Service.** Flooding gossip with invalid messages is mitigated by size bounds (10MB max) and rate limiting (10 msg/sec per peer), but full DoS resistance requires stake-based reputation.



**A3. Validator Compromise.** If  $\geq T$  Hyperlane validators (or the LayerZero endpoint) are compromised, fraudulent events can be injected. This is a limitation inherited from the cross-chain messaging layer, not Igra itself.

**A4. Key Exposure.** Signing keys exist in process memory. Physical or remote access to the signer process during signing operations could expose keys. HSM integration would mitigate this.

**A5. Storage Tampering.** RocksDB files are not encrypted or integrity-checked. Filesystem-level access could corrupt or manipulate stored state. This is mitigated by operating system access controls.

## 5 Reference Implementation

We validate the theoretical model through an implementation on Kaspas, a high-throughput UTXO blockchain.

### 5.1 Implementation Mapping

| Theoretical Concept  | Implementation                      |
|----------------------|-------------------------------------|
| Event identifier     | BLAKE3 with domain "igra:event:v1:" |
| Template format      | PSKT (Kaspa PSBT variant), JSON     |
| Template identifier  | BLAKE3 of Borsh-serialized skeleton |
| Gossip transport     | Iroh gossip protocol                |
| Signature scheme     | secp256k1 Schnorr (per-input)       |
| Storage (production) | RocksDB                             |
| Storage (testing)    | In-memory HashMap                   |
| Key management       | BIP39/BIP32 HD + XChaCha20Poly1305  |

Table 1: Mapping from theoretical concepts to implementation

### 5.2 Event Structure

```
struct Event {
    external_id: [u8; 32],
    source: SourceType,
    destination: ScriptPubKey,
    amount: u64,
}
```

Event identifier computation:

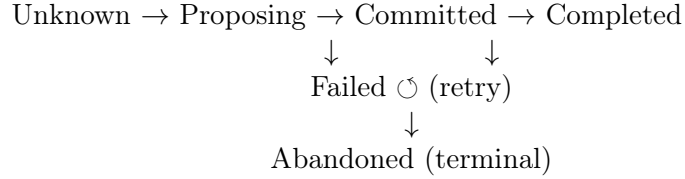
1. Domain-separate with "igra:event:v1:"
2. Encode fields (concatenated)
3. Hash with BLAKE3

### 5.3 Template Hashing

Templates use PSKT format:

1. Store/transmit as JSON-serialized PSKT
2. To hash: extract unsigned transaction  $\rightarrow$  serialize with Borsh  $\rightarrow$  BLAKE3

### 5.4 Phase State Machine



### 5.5 Validation Against Theoretical Model

| Property        | Theoretical                     | Implementation                       | Valid |
|-----------------|---------------------------------|--------------------------------------|-------|
| Unique vote     | Key $(e, r, p)$                 | RocksDB key                          | ✓     |
| Selection       | Algorithm 3.3                   | <code>selection.rs</code>            | ✓     |
| CRDT merge      | Set $\cup$ , LWW                | <code>rocks/event_store.rs</code>    | ✓     |
| Template hash   | $\mathcal{H}(\text{canonical})$ | <code>multisig.rs</code>             | ✓     |
| Validator check | m-of-n                          | <code>validation/hyperlane.rs</code> | ✓     |
| Peer auth       | Ed25519                         | <code>iroh/filtering.rs</code>       | ✓     |

Table 2: Validation of implementation against theory

### 5.6 Performance

**Testnet (preliminary):**

- Commit latency: 200–800ms (3-node)
- Retry rate: 5–15% (simulated UTXO divergence)
- Bandwidth:  $\sim$ 2KB/proposal,  $\sim$ 500B/signature

**Complexity:**

- Messages:  $O(N^2)$ /round
- Storage:  $O(E \cdot R \cdot N)$  proposals,  $O(E \cdot I \cdot N)$  signatures

## 6 Discussion

### 6.1 Comparison to Alternatives

**Leader-based consensus (Raft, Paxos).** Requires leader election; single point of failure. Igra is leaderless.

**BFT consensus (PBFT, Tendermint).** Handles Byzantine faults but requires  $N \geq 3f + 1$ . Igra achieves safety under crash faults with  $N \geq 2f + 1$ .

**Optimistic concurrency.** Signing without coordination risks double-spend. Igra trades latency for safety.

## 6.2 Limitations and Extensions

**Byzantine tolerance.** Full BFT requires lock certificates and explicit slashing.

**Chain abstraction.** Generalizing to Bitcoin requires abstract template interface and PSBT adapters.

**Interactive ceremonies.** MuSig2/FROST require extended CRDT for interactive artifacts.

**Hardware security.** HSM integration would prevent key exposure.

## 7 Related Work

**Byzantine consensus.** PBFT [10] and Tendermint [11] provide BFT safety but target blockchain consensus, not threshold signing.

**CRDTs.** Shapiro et al. [12] formalized conflict-free replication. Igra uses G-Set and LWW-Register.

**Threshold signatures.** MuSig2 [13], FROST [14], GG20 [15] address cryptographic signing. Igra coordinates *when* to sign.

**MPC.** CHURP [16] and Taurus [17] handle key generation. Igra focuses on coordination given existing keys.

## 8 Conclusion

We presented Igra, a leaderless protocol for safe threshold signing on UTXO blockchains. The protocol guarantees at most one transaction is signed per event through two-phase quorum voting and CRDT signature propagation. Formal proofs establish safety and convergence under crash faults. A Kaspera implementation validates the model.

The trust model is explicit: external validators authenticate events, industry-standard key derivation manages signing keys, and authenticated gossip coordinates peers. Physical security is assumed but not enforced.

Future work includes BFT extensions, Bitcoin support, HSM integration, and interactive ceremonies.

## Acknowledgments

[To be filled for publication]

## References

- [1] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008. <https://bitcoin.org/bitcoin.pdf>
- [2] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Michael Peyton Jones, and Philip Wadler. The extended UTXO model. In *Financial Cryptography Workshops*, 2020.
- [3] Evaldas Drasutis. UTXO chains: Programmability limitations and L2 solutions. *Medium*, 2024. <https://medium.com/@lunfardo/utxo-chains-part-4-499d12734abc>
- [4] Bitcoin Optech. Covenants. <https://bitcoinops.org/en/topics/covenants/>
- [5] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm (Raft). In *USENIX ATC*, 2014.
- [6] Leslie Lamport. The part-time parliament (Paxos). *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
- [7] Jianyu Niu et al. EquinoxBFT: BFT consensus for blockchain emergency governance. In *Blockchain and Trustworthy Systems*, 2024.
- [8] Bitcoin Optech. Threshold signatures. <https://bitcoinops.org/en/topics/threshold-signature/>
- [9] John Adler et al. Optimistic rollups. *Ethereum Research*, 2019.
- [10] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *OSDI*, 1999.
- [11] Ethan Buchman. Tendermint: Byzantine fault tolerance in the age of blockchains. Master’s thesis, University of Guelph, 2016.
- [12] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, 2011.
- [13] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In *CRYPTO*, 2021.
- [14] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *Selected Areas in Cryptography*, 2020.
- [15] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In *ACM CCS*, 2018.
- [16] Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. CHURP: Dynamic-committee proactive secret sharing. In *ACM CCS*, 2019.
- [17] Fabrice Benhamouda, Tancrède Lepoint, Julian Loss, Michele Orrù, and Mariana Raykova. Taurus: Sharing and outsourcing threshold cryptography. In *USENIX Security*, 2022.

| Symbol        | Meaning                     |
|---------------|-----------------------------|
| $N$           | Number of signers           |
| $m$           | Signature threshold         |
| $q$           | Commit quorum ( $q > N/2$ ) |
| $T$           | Validator threshold         |
| $M$           | Total validators            |
| $\mathcal{P}$ | Peer identifier set         |
| $\mathcal{H}$ | BLAKE3 hash                 |
| $e$           | Event                       |
| $\tau$        | Transaction template        |
| $r$           | Round number                |
| $\Sigma$      | Signature set (G-Set)       |
| $C$           | Completion (LWW-Register)   |

Table 3: Notation reference

## A Notation

## B Implementation Reference

Reference implementation: `rusty-kaspa/wallet/igra`

### Core modules:

- Event/template hashing: `igra-core/src/domain/hashes.rs`
- Selection: `igra-core/src/domain/coordination/selection.rs`
- Phase machine: `igra-core/src/domain/coordination/phase.rs`
- Storage (prod): `igra-core/src/infrastructure/storage/rocks/`
- Storage (test): `igra-core/src/infrastructure/storage/memory.rs`
- Gossip: `igra-core/src/infrastructure/transport/iroh/`
- Validation: `igra-core/src/domain/validation/`
- Key mgmt: `igra-core/src/foundation/hd.rs`

### Verification:

- Unit tests (determinism)
- Integration tests (UTXO divergence, validator checks)
- Testnet deployment